Active Object

Active Object · Serialize ledger mutations on one servant · Build with Waqar

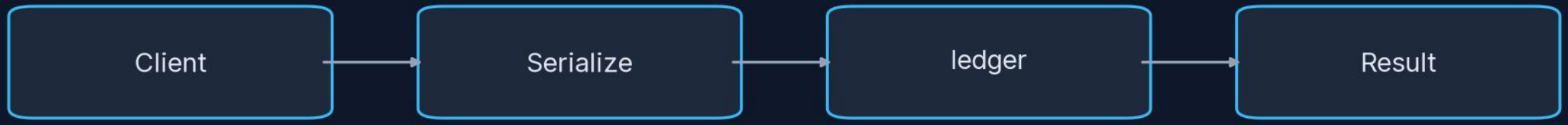
Today's Hook

- Today's force: Serialize ledger mutations on one servant
- We earn Active Object by feeling the naive pain first.
- Demo main prints a real JDK 25 trace (happy + failure).
- Key types: ActiveLedger

The Problem

- Scenario: Serialize ledger mutations on one servant
- Constraints: solve «Serialize ledger mutations on one servant» with Active Object — keep the demo runnable and the failure path observable.
- Naive pain: Naive version fails to cleanly solve: Serialize ledger mutations on one servant
- Success looks like: bal=15

Structure



Active Object: Serialize ledger mutations on one servant

Real-World Scenario

- Running example: Serialize ledger mutations on one servant
- Key types: ActiveLedger
- Happy-path stdout cue: bal=15
- Failure-path stdout cue: Failure path is asserted in ProblemTest (injected fault / invalid input)

Naive Code

```
package com.buildwithwaqar.patterns.concurrency.activeobject.problem;

import java.util.*;

public final class NaiveLedger {
  private int bal; public void deposit(int c){ bal+=c; } public int bal(){ return bal; }
}
```

Why Naive Hurts

- Naive version fails to cleanly solve: Serialize ledger mutations on one servant
- Without Active Object, call sites absorb the wrong details.
- Failure mode to show on camera: Failure path is asserted in ProblemTest (injected fault / invalid input)
- New requirements force shotgun edits across the naive code.

Solution

```java
package com.buildwithwaqar.patterns.concurrency.activeobject.problem;

import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;
import java.util.concurrent.locks.*;
import java.util.function.*;

final class ActiveLedger implements AutoCloseable {
  private int bal; private final ExecutorService servant=Executors.newSingleThreadExecutor();
  CompletableFuture<Void> deposit(int c){ return CompletableFuture.runAsync(() -> { bal+=c; }, servant); }
  CompletableFuture<Integer> balance(){ return CompletableFuture.supplyAsync(() -> bal, servant); }
  public void close(){ servant.shutdown(); }
}
```

Runnable main

```
package com.buildwithwaqar.patterns.concurrency.activeobject.problem;
import java.util.*; import java.util.concurrent.*; import java.util.concurrent.atomic.*;
public final class ProblemDemo {
  public static void main(String[] args) throws Exception {
    System.out.println("=== Active Object: serialize ledger ===");
    System.out.println("--- naive ---"); var n=new NaiveLedger(); n.deposit(5); System.out.println(n.bal());
System.out.println("--- active object ---");
try (var a=new ActiveLedger()) {
  a.deposit(10).join(); a.deposit(5).join();
  System.out.println("bal="+a.balance().join());
}
  }
}
```

Walkthrough Output

```
=== Active Object: serialize ledger ===
--- naive ---
5
--- active object ---
bal=15
```

Step-by-Step Trace

- Match stdout to stages — look for: bal=15
- Then the injected failure: Failure path is asserted in ProblemTest (injected fault / invalid input)
- Re-run: java -cp … 39-active-object ProblemDemo (see episode folder).
- Pause the video and predict the next printed line.

Modern Java 25 / Zazr

```
package com.buildwithwaqar.patterns.concurrency.activeobject.problem;

import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;
import java.util.concurrent.locks.*;
import java.util.function.*;

final class ActiveLedger implements AutoCloseable {
  private int bal; private final ExecutorService servant=Executors.newSingleThreadExecutor();
  CompletableFuture<Void> deposit(int c){ return CompletableFuture.runAsync(() -> { bal+=c; }, servant); }
  CompletableFuture<Integer> balance(){ return CompletableFuture.supplyAsync(() -> bal, servant); }
  public void close(){ servant.shutdown(); }
}
```

Trade-offs

- Use Active Object when «Serialize ledger mutations on one servant» repeats and a simpler feature does not erase it.
- Refuse extra types when a sealed switch or a function value already suffices.
- Teaching demos omit production metrics, retries, and multi-region clocks.
- Interview soundbite: state the force behind Active Object in product language.

Related Patterns

- Name the pattern usually confused with Active Object and separate the forces.
- Sketch the roles for «Serialize ledger mutations on one servant» from memory in sixty seconds.
- Point to the next episode in this season on the playlist map.
- Link this episode's series/ folder in the YouTube description.

Recap

- Problem: Serialize ledger mutations on one servant
- Naive pain → Active Object boundary → Demo stdout.
- Remember the failure cue: Failure path is asserted in ProblemTest (injected fault / invalid input)
- Break a ProblemTest assertion, watch it fail, restore it.